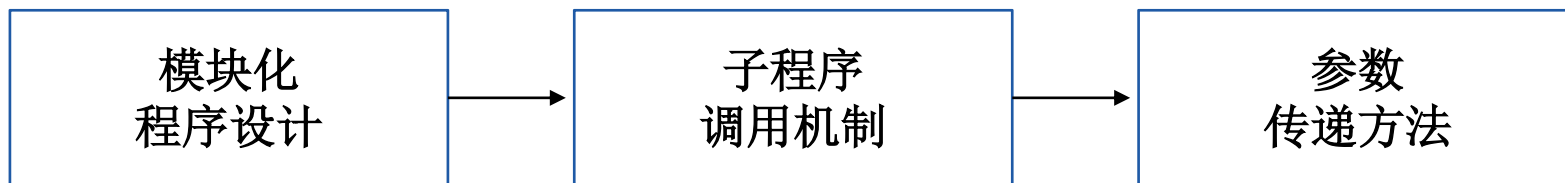


模块化程序设计



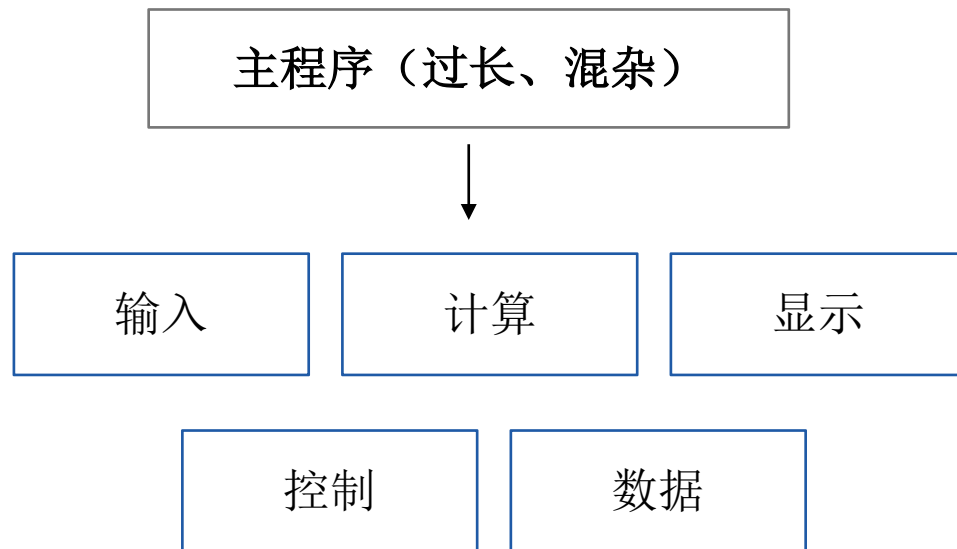
目录



- 第一章 子程序设计
- 第二章 CALL / RET 原理
- 第三章 子程序调用示例分析
- 第四章 寄存器与共享变量传参
- 第五章 堆栈传参与栈帧
- 第六章 多模块程序与宏

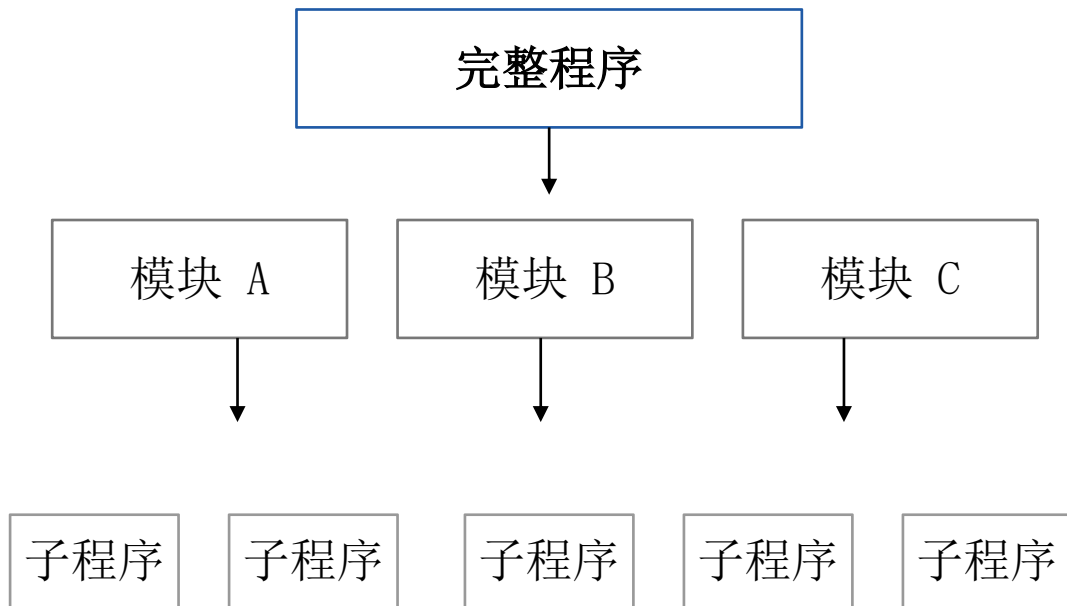
为什么要学习模块化程序设计

- 程序规模一变大，主程序会越来越长。
- 同样功能如果反复复制，修改会非常麻烦。
- 调试时难以快速定位错误位置。
- 模块化的目标：分功能、可复用、易维护、便于扩展。



主程序、子程序、模块，三者是什么关系

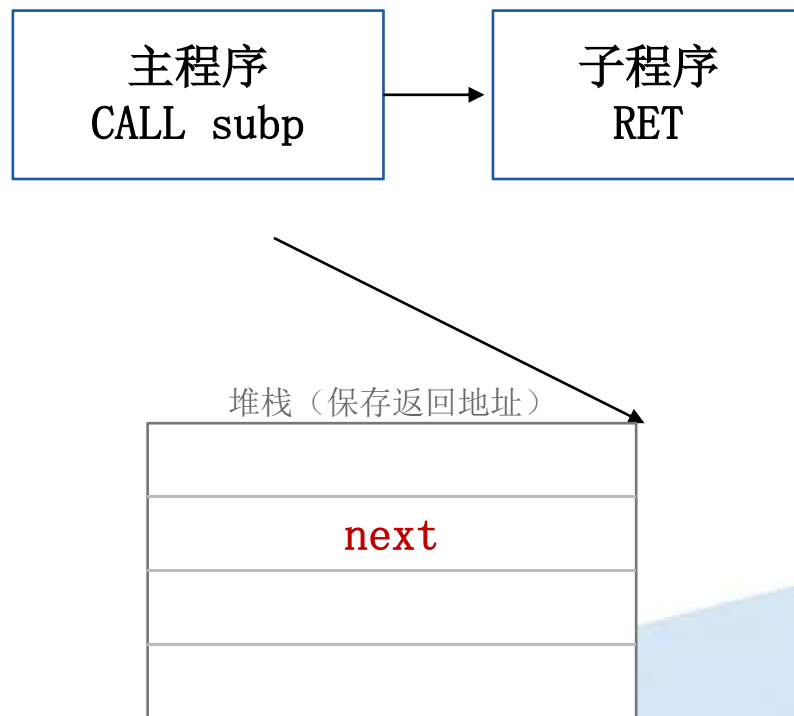
- 主程序：负责整体控制，决定先做什么、后做什么。
- 子程序：负责完成某个具体功能。
- 模块：把若干相关代码组织在同一个文件或单元中。
- 层次关系：代码语句 → 子程序 → 模块 → 完整程序。



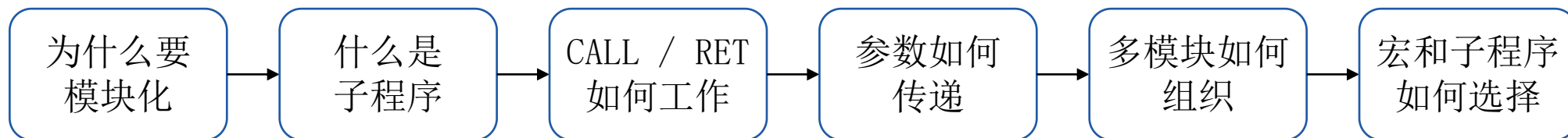
子程序调用的本质：跳过去，还能再回来



- 主程序执行到 CALL 时，不是简单跳转。
- CALL 的本质：先保存返回地址，再转去执行子程序。
- RET 的本质：取回返回地址，再回到主程序继续执行。
- 因此，子程序不是“复制代码”，而是“跳转 + 返回”。



本章学习路线图



这节课不是只学两条指令，而是在学“大程序怎么写”。

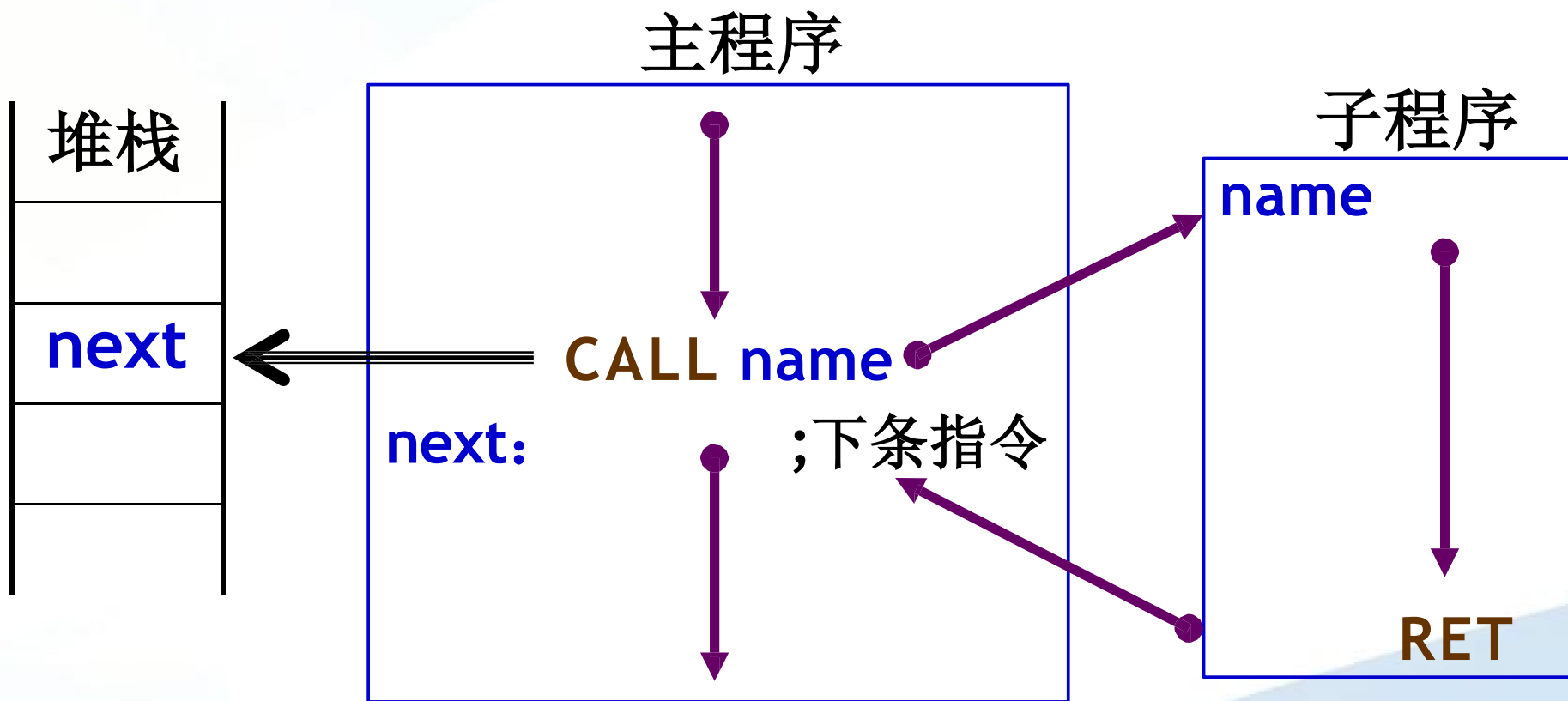


汇编语言程序设计

1. 子程序设计



RET指令返回主程序，CALL指令调用子程序



标准子程序框架：逐行看懂它在做什么



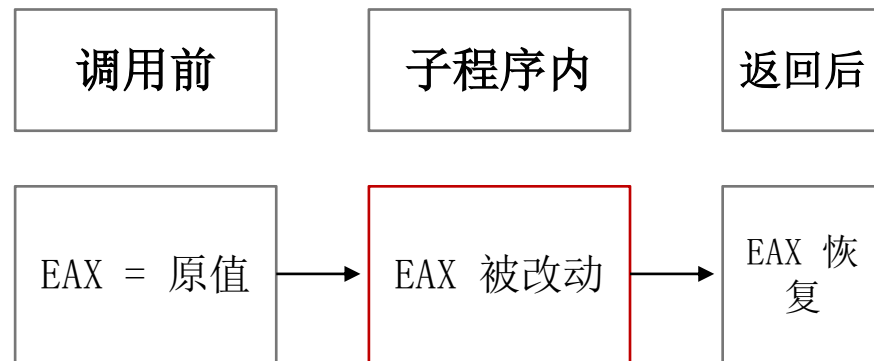
```
name proc  
    push ...  
    ; 子程序体  
    pop ...  
    ret  
name endp
```

- name proc: 子程序开始，定义名字。
- push ...: 保护可能被修改的寄存器。
- 子程序体: 完成实际功能。
- pop ...: 恢复寄存器。
- ret: 返回主程序。
- name endp: 子程序结束。



什么叫保护现场，什么叫恢复现场

- 主程序在调用前，很多寄存器里已经有重要数据。
- 子程序执行时可能会改动这些寄存器。
- 如果不保护，主程序回来后原数据可能被破坏。
- 常见做法：进入子程序先 push，返回前再 pop。



保护现场：先保存旧值 | 恢复现场：结束时还原旧值



过程定义伪指令

- MASM利用过程定义伪指令获得子程序信息

过程名 **PROC**

;过程体

过程名 **ENDP**

;过程名为符合语法的标识符

MASM会根据存储模型等信息确定子程序的远近调用，并相应产生调用、返回指令



子程序框架

标识符

proc	;过程定义（子程序开始）
push ...1	;保护寄存器
push ...2	
	;子程序体
pop ...2	;恢复寄存器
pop ...1	
ret	;子程序返回
endp	;过程（子程序）结束

标识符



子程序框架（堆栈传递参数）

标识符

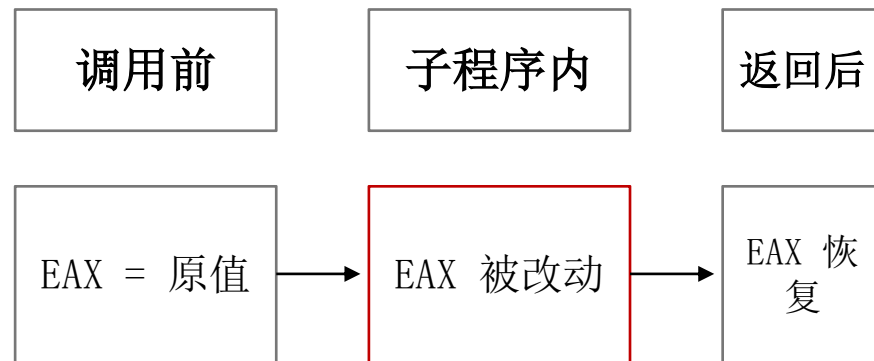
proc	;过程定义（子程序开始）
push ebp	; 堆栈传递入口参数
mov ebp,esp	
push ...1	;保护寄存器
push ...2	
	;子程序体
pop ...2	;恢复寄存器
pop ...1	
pop ebp	;子程序返回
ret	
endp	:过程（子程序）结束

标识符

保护寄存器：保护现场，恢复现场



- 主程序在调用前，很多寄存器里已经有重要数据。
- 子程序执行时可能会改动这些寄存器。
- 如果不保护，主程序回来后原数据可能被破坏。
- 常见做法：进入子程序先 push，返回前再 pop。



保护现场：先保存旧值 | 恢复现场：结束时还原旧值

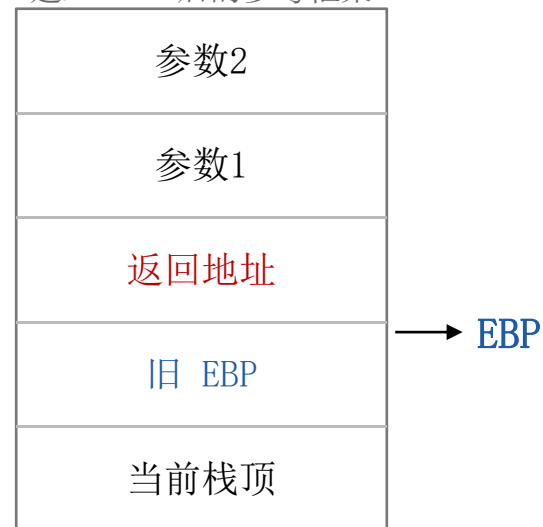


为什么堆栈传参时要先建立 EBP

```
push ebp  
mov  ebp, esp
```

- ESP 会随着 push / pop 不断变化。
- 如果直接用 ESP 找参数，位置容易变化。
- 建立 EBP 后，当前栈顶位置就有了一个固定参考点。
- 后面便可以稳定地用 $[ebp+8]$ 、 $[ebp+12]$ 访问参数。

建立 EBP 后的参考框架





案例：回车换行功能

mov al,0dh ;输出回车字符, 12

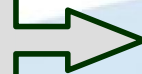
call dispc

mov al,0ah ;输出换行字符, 10

call dispc

;子程序中调用子程序, 实现子程序嵌套

编写成子程序





回车换行子程序

dpcrlf proc

push eax;保存EAX寄存器值到堆栈
mov al, 0dh;将回车符(0DH)送入AL寄存器
call dispc ;并调用显示字符子程序
mov al, 0ah;将换行符(0AH)送入AL寄存器
call dispc ;调用显示字符子程序
pop eax;恢复EAX寄存器值

ret ;子程序返回
dpcrlf endp



子程序设计

➤ 子程序的编写方法与主程序一样

➤ 但需要留意几个问题：

应有完整的注释

- ✓ RET指令返回主程序，CALL指令调用子程序
- ✓ 利用过程定义，获得子程序名和调用属性
- ✓ 压入和弹出操作要成对使用，保持堆栈平衡
- ✓ 子程序开始保护寄存器，返回前相应恢复
- ✓ 安排在代码段的主程序之外
- ✓ 子程序允许嵌套和递归

难点是参数传递



参数传递

- 主程序与子程序间通过参数传递建立联系
 - ▶ 入口参数（输入参数）：主程序→子程序
 - ▶ 出口参数（输出参数）：子程序→主程序
- 参数的具体内容
 - ▶ 数据本身（传递数值）
 - ▶ 数据的存储地址
（传递地址，传递引用）



参数传递方法

- ✓通用寄存器
- ✓共享变量
- ✓堆栈

传值与传地址，到底有什么区别



传值：把数据本身交给子程序

- 适合：整数、字符、少量简单数据。
- 本质：直接给“东西”。

直接给苹果

传地址：把数据所在位置交给子程序

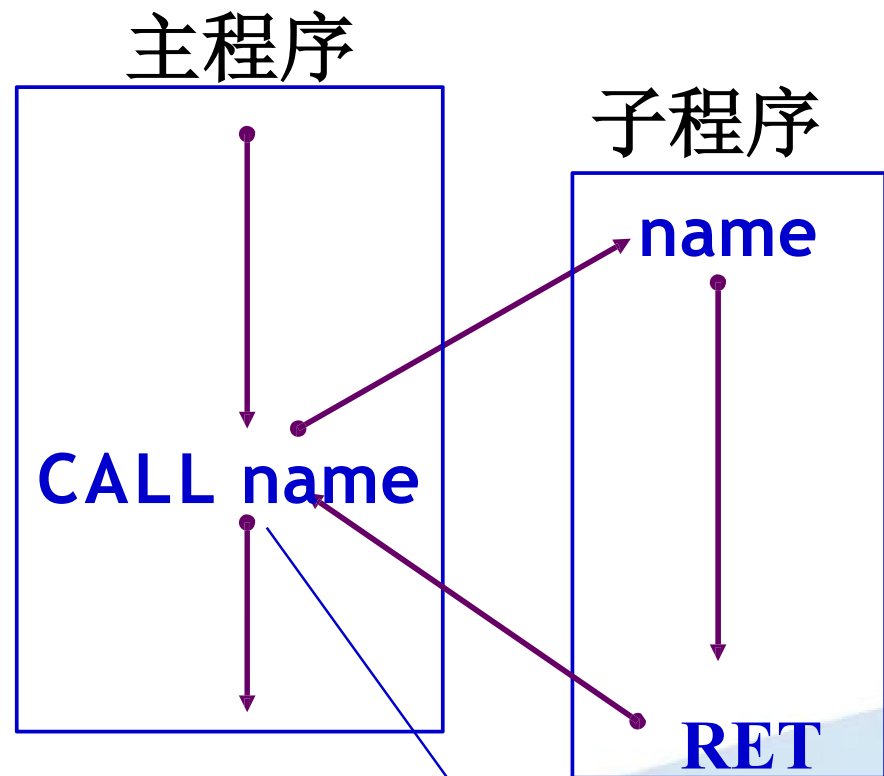
- 适合：数组、字符串、大块数据。
- 本质：给“位置”，再由子程序间接访问。

给仓库地址

CALL和 RET指令

子程序指令

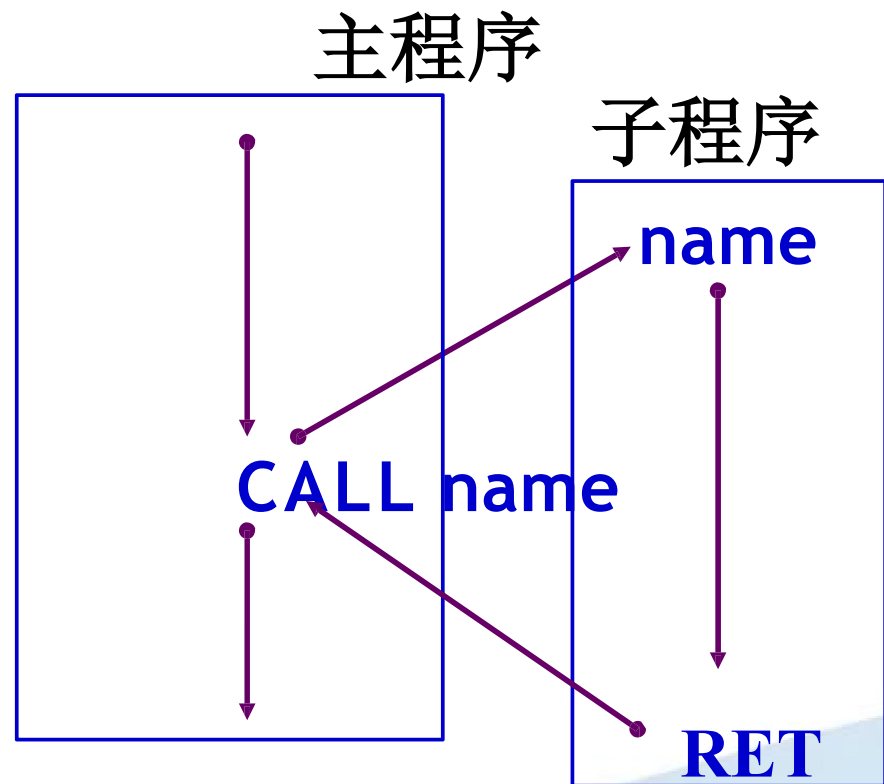
- 主程序 (调用程序)
 - 执行调用指令CALL
调用子程序
- 子程序 (被调用程序)
 - 执行返回指令RET
返回主程序



回到CALL指令
的下条指令处

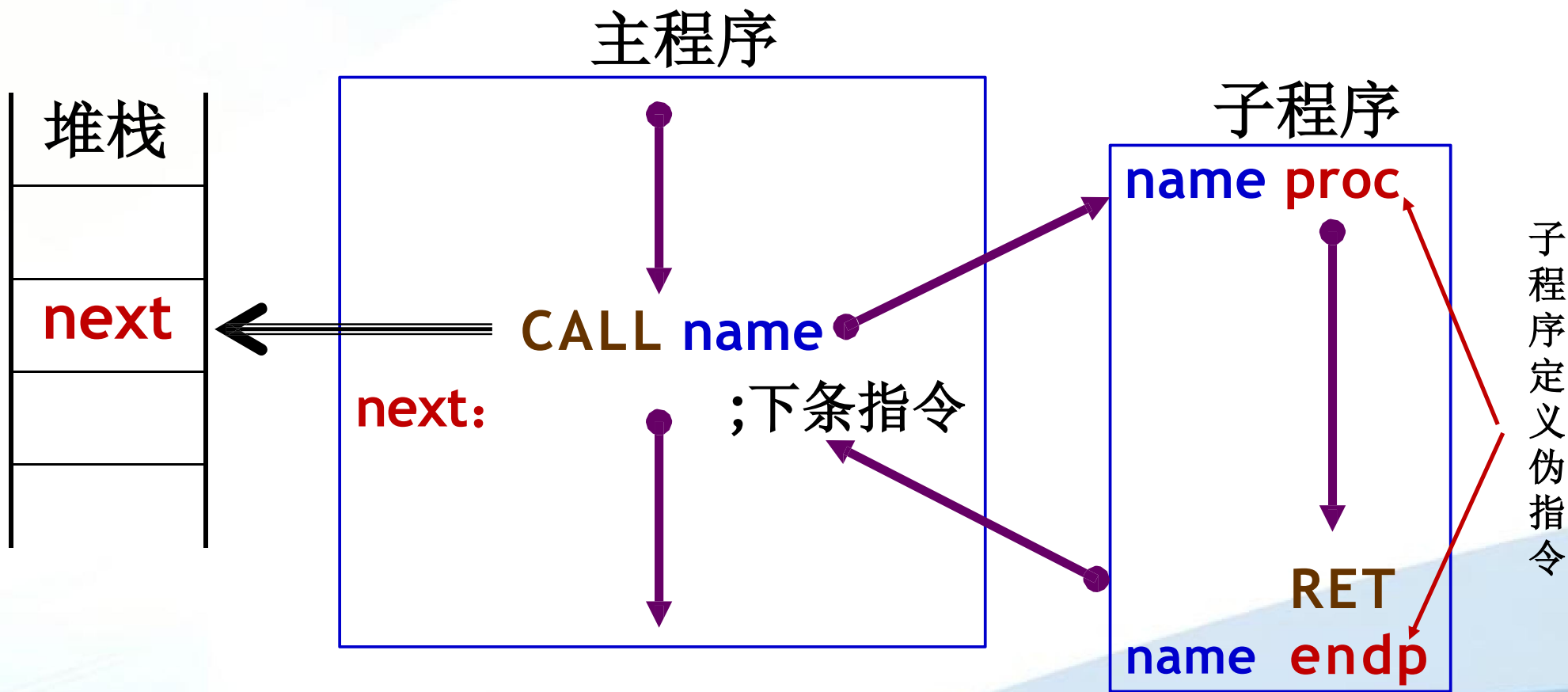
CALL和RET指令的功能

- CALL指令用在主程序中实现子程序的调用
 - ▶ 功能1：入栈返回地址
 - ▶ 功能2：转移到目标地址
- RET指令用在子程序结束实现返回主程序
 - ▶ 功能1：弹出返回地址
 - ▶ 功能2：转移到返回地址





CALL和RET利用堆栈保存返回地址





子程序调用指令CALL

- CALL指令用在主程序中，实现子程序的调用
 - ▶ 功能1：将下条指令的地址压入堆栈（顶部）
 - ▶ 功能2：转移到目标地址

CALL label ;调用标号指定的子程序

CALL reg32/reg16 ;调用寄存器指定地址的子程序

CALL mem48/mem32/mem16 ;调用存储单元指定地址的子程序

CALL分成段内调用（近调用）和段间调用（远调用）
目标地址支持直接寻址、相对寻址或间接寻址



重点：子程序返回指令RET

- RET指令用在子程序结束，实现返回主程序
 - ▶ 功能1：从当前堆栈顶部弹出内容作为返回地址
 - ▶ 功能2：转移到返回地址

RET ;无参数返回：出栈返回地址

RET i16 ;有参数返回：出栈返回地址， $ESP \leftarrow ESP + i16$

调用、返回有段内near和段间far的区别
但是，CALL和RET指令助记符没有区别



重点：段内CALL和RET指令

CALL label

next:

;相当于如下两条指令功能

push next

;(1) 入栈返回地址: $ESP=ESP-4$, $SS:[ESP]=EIP$

jmp label

;(2) 转移目标地址: $EIP=EIP+偏移量$

next



RET

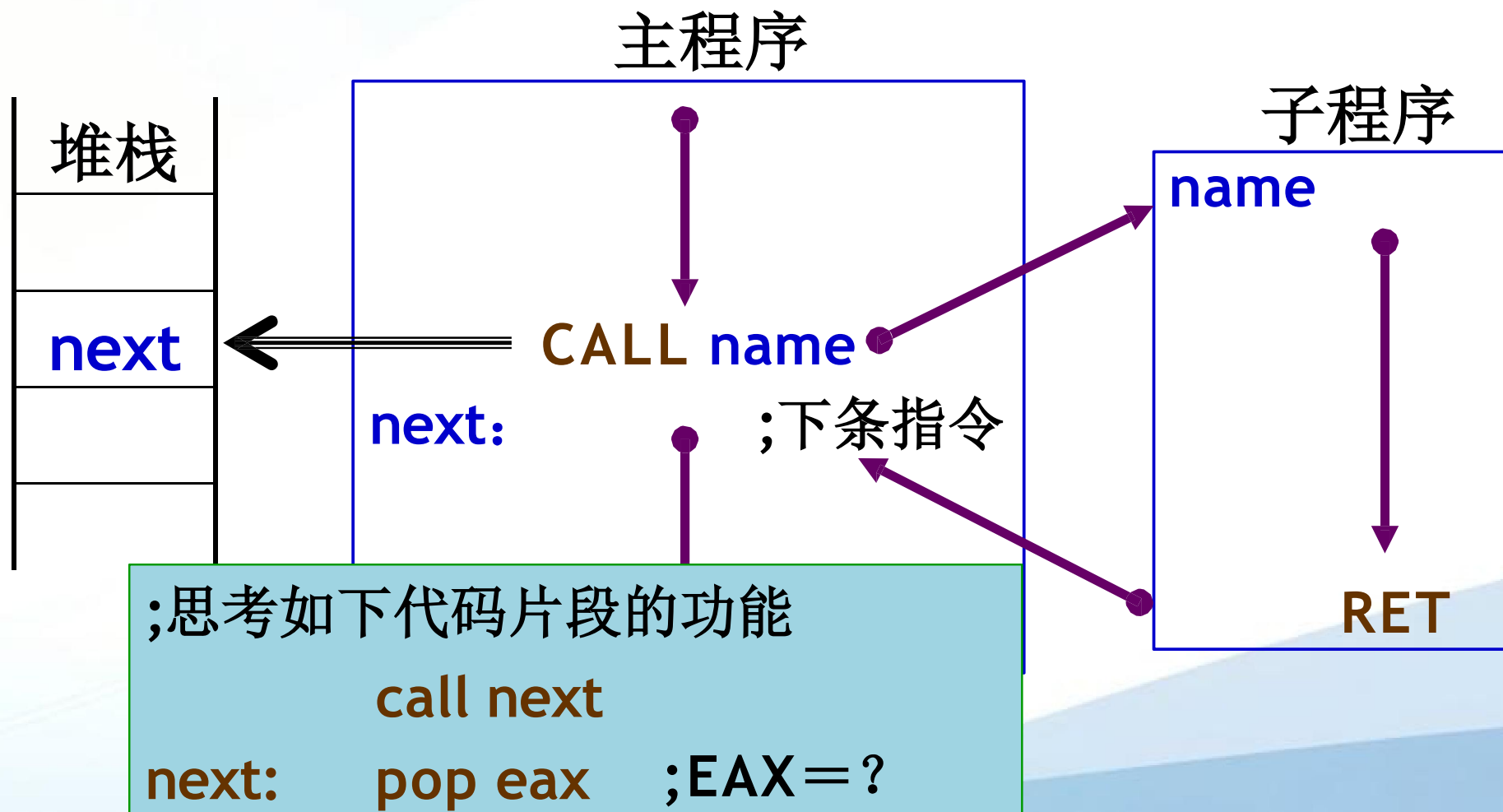
;栈顶数据出栈到指令指针寄存器EIP

;(1) $EIP=SS:[ESP]$, $ESP=ESP+4$

;(2) 数据进入EIP, 就作为下条要执行指令的地址



CALL的调用和RET的返回



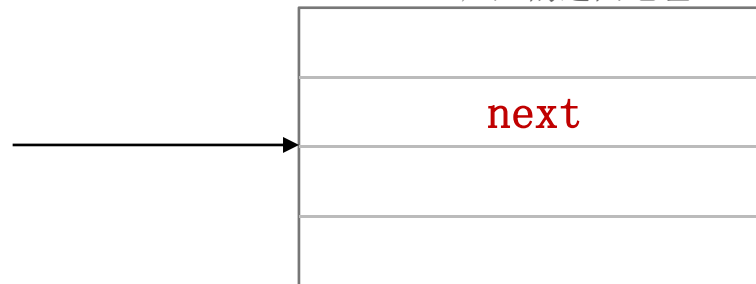
思考题解析: call next / pop eax 为什么成立



```
call next
next:
    pop eax
```

- call next 先把 next 的地址压入堆栈。
- 然后控制转到 next。
- pop eax 把这个地址弹到 EAX。
- 所以 EAX 中得到的是标号 next 的地址。

CALL 压入的返回地址





这一小题真正说明了什么

- 返回地址本身也是一段数据。
- 这段数据被保存在堆栈里。
- 只要能访问栈，就能观察到返回地址。
- 因此，程序控制流和堆栈结构是紧密相关的。
- 理解 CALL / RET，必须从“堆栈视角”切入。

控制流：主程序 → 子程序
→ 主程序

数据流：返回地址 ↔ 堆
栈

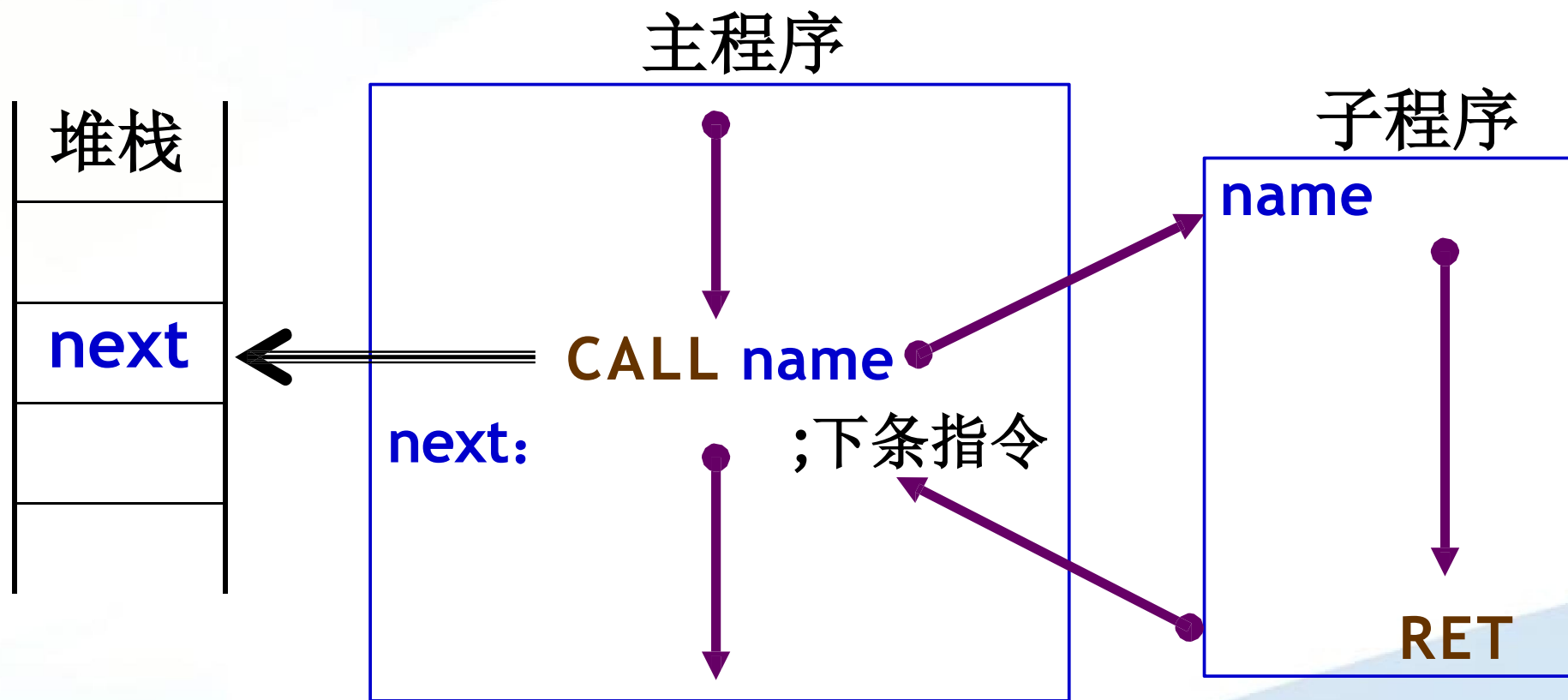
二者在调用过程中交织在一
起



汇编语言程序设计

子程序调用

子程序调用CALL和返回RET指令





子程序调用程序—1

;列表文件的地址

00000000

00000005

0000000A

0000000F ←

00000014 ←

00000019

;代码段，主程序

mov eax,1

mov ebp,5

call subp

ret p1: mov ecx,3

ret p2: mov edx,4

call disprd

;子程序调用



重点：段内CALL和RET指令

CALL label

next:

;相当于如下两条指令功能

push next

;(1) 入栈返回地址: $ESP=ESP-4$, $SS:[ESP]=EIP$

jmp label

;(2) 转移目标地址: $EIP=EIP+偏移量$

next



RET

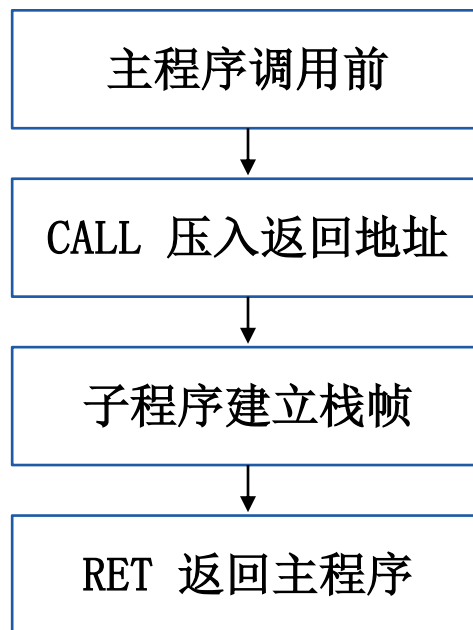
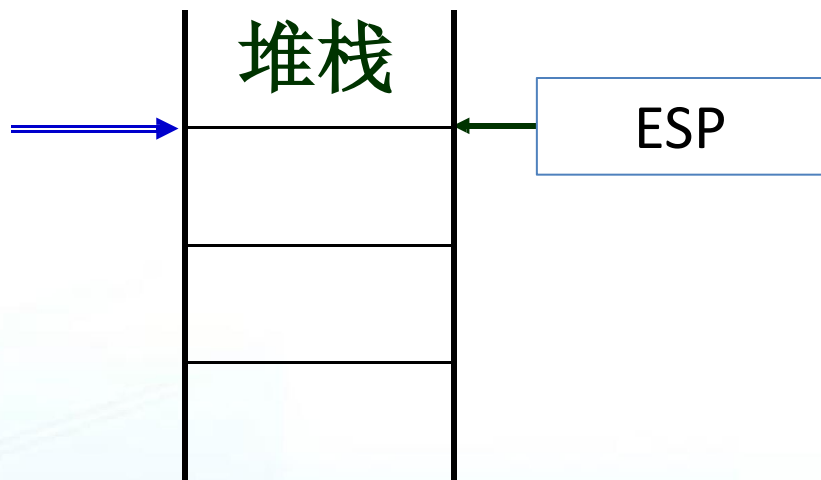
;栈顶数据出栈到指令指针寄存器EIP

;(1) $EIP=SS:[ESP]$, $ESP=ESP+4$

;(2) 数据进入EIP, 就作为下条要执行指令的地址

子程序调用

- 主程序先执行: `mov eax,1; mov ebp,5; call subp`。
- 调用时, 返回地址被压入堆栈。
- 进入 `subp` 后, `push ebp; mov ebp,esp`, 建立栈帧。
- 随后子程序读取 `[ebp+4]`, 取得返回地址。
- 返回前恢复 `ebp`, 再执行 `ret`。
- 主程序回到调用点后继续执行后续语句。



子程序调用

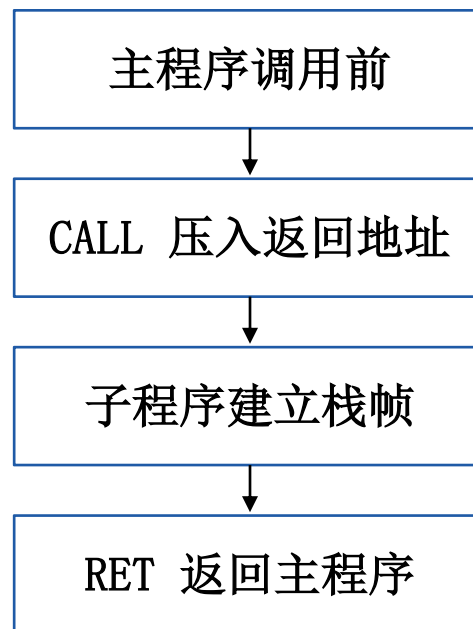
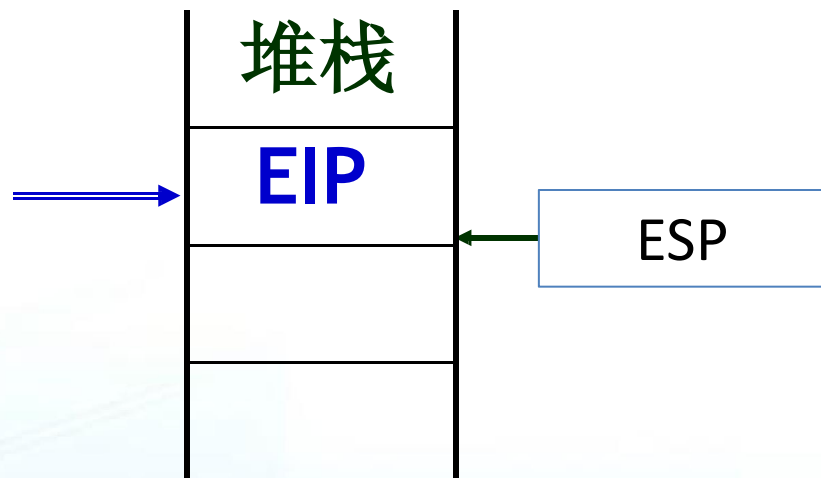


1. 主程序开始执行:

mov eax,1 → EAX = 1

mov ebp,5 → EBP = 5

call subp → 将返回地址 (retip1的地址) 压栈, 即 [esp], 跳转到子程序subp





子程序调用程序—2

;代码段，子程序

subp

proc

;过程定义，过程名为subp

push ebp

mov ebp,esp

mov esi,[ebp+4]

;ESI=CALL下条指令(标号RETP1)偏移地址

push ebp: 入栈，retpl的地址存在[esp+4]



子程序调用程序—2

;代码段，子程序

subp

proc

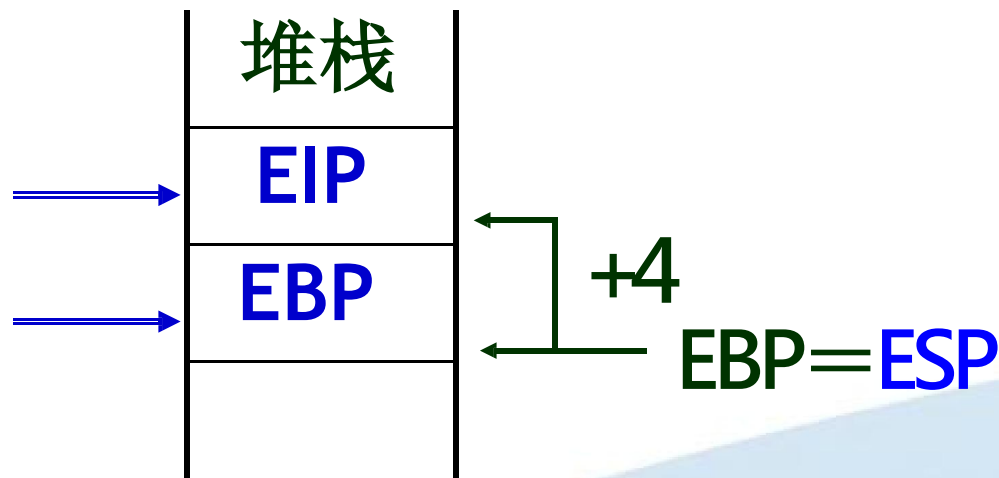
;过程定义，过程名为subp

push ebp

mov ebp,esp

mov esi,[ebp+4]

CALL压入的返回地址
子程序压入的EBP



mov esi, [ebp+4];从栈中获取返回地址（ret p1的地址）存入ESI，ESI=offset ret p1



子程序调用程序—3

mov edi,offset retp2

mov ebx,2

pop ebp

ret

subp endp

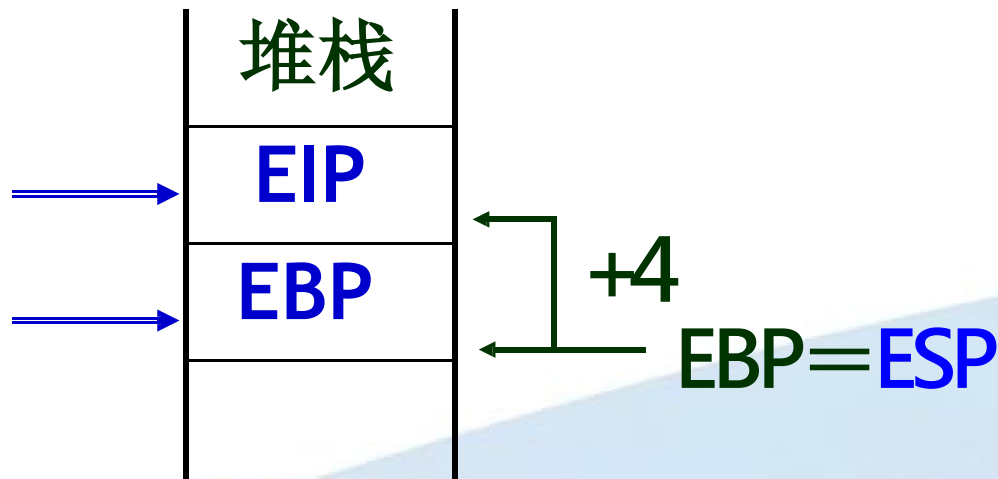
;弹出堆栈，保持堆栈平衡

;子程序返回

;过程结束

CALL压入的返回地址

子程序压入的EBP



mov edi,offset retp2 -> edi=offset retp2

pop ebp-> 恢复EBP=5

ret-> 返回主程序retp1处



主程序和子程序

;代码段，主程序

mov eax,1

mov ebp,5

call subp

ret p1:

mov ecx,3

ret p2:

mov edx,4

call disprd

subp

;代码段，子程序

proc

push ebp

mov ebp,esp

mov esi,[ebp+4]

mov edi,offset ret p2

mov ebx,2

pop ebp

ret

endp

subp



程序运行结果

;列表文件的地址

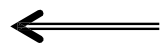
00000000

00000005

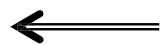
0000000A

0000000F

00000014



retpl:



retpl2:

;代码段, 主程序

mov eax,1

mov ebp,5

call subp

mov ecx,3

mov edx,4

ESI=offset retp1

EDI=offset retp2

```
D:\MASM>eg0501.exe
```

```
EAX=00000001, EBX=00000002, ECX=00000003, EDX=00000004
```

```
ESI=0040101F, EDI=00401024, EBP=00000005, ESP=0012FFC4
```

→ 本程序运行时, 起始地址: 00401010H, 需要加上代码段地址



思考：子程序调用程序—3a

mov edi,offset retp2

; EDI = 标号RETP2的偏移地址

mov ebx,2

pop ebp

ret

subp endp

CALL压入的返回地址

子程序压入的EBP

ret地址获取的地址是[ebp+4]里面存的变成了offset retp2，因此返回之后不执行retp1,所以ecx!=3

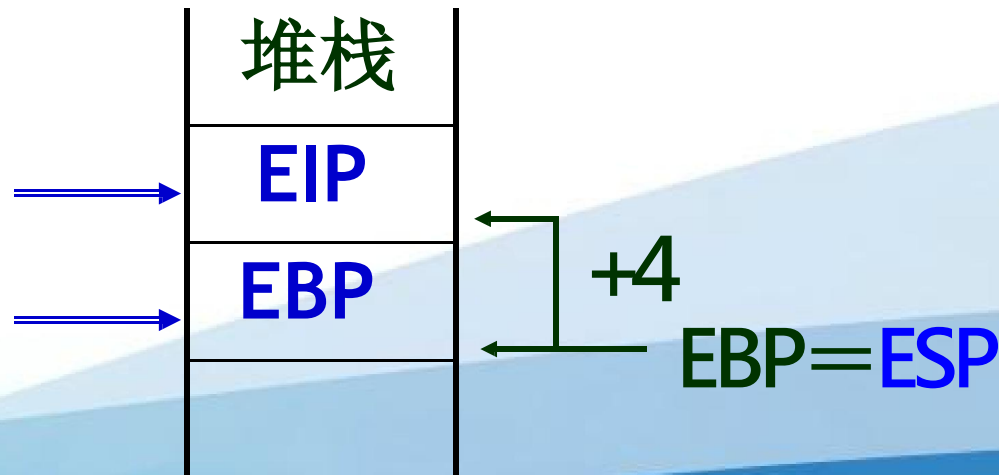
MOV [EBP+4],EDI, 导致执行ret的返回地址
[ebp+4]变成了offset retp2

MOV [EBP+4],EDI

;弹出堆栈，保持堆栈平衡

;子程序返回

;过程结束





程序运行结果a

;列表文件的地址

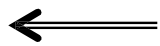
00000000

00000005

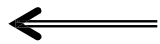
0000000A

0000000F

00000014



ret p1:



ret p2:

;代码段，主程序

mov eax,1

mov ebp,5

call subp

mov ecx,3

mov edx,4

ret地址获取的地址是[ebp+4]
里面存的变成了offset ret p2,
因此返回之后不执行ret p1,所以ecx!=3

D:\MASM>eg0501a.exe

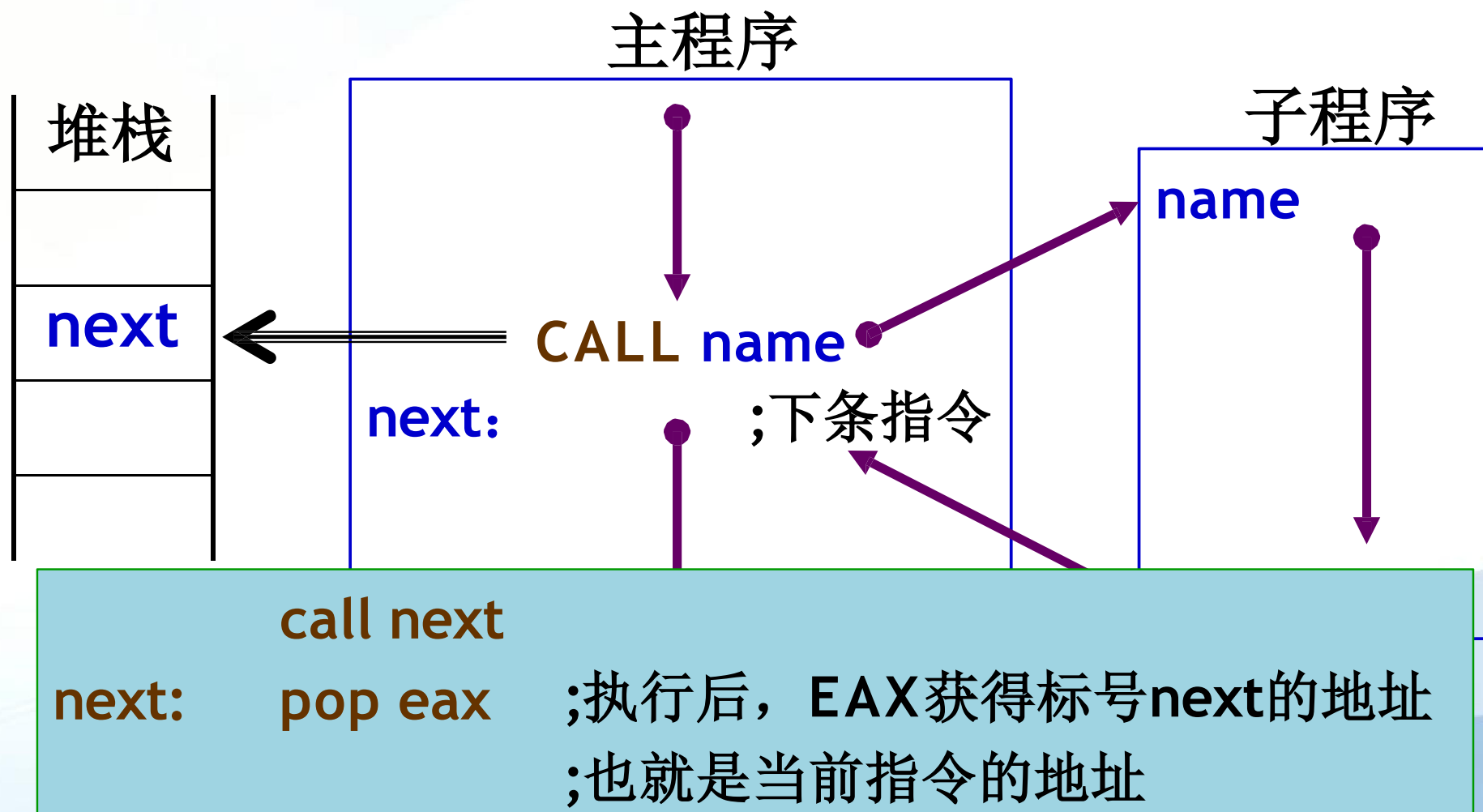
EAX=00000001, EBX=00000002, ECX=0012FFB0, EDX=00000004

ESI=0040101F, EDI=00401024, EBP=00000005, ESP=0012FFC4

→ 本程序运行时，起始地址：00401010H



CALL和RET利用堆栈保存返回地址



寄存器传递参数



寄存器传递参数

- 最简单和常用的参数传递方法
- 把参数存于约定的寄存器
 - 少量数据直接传递数值
 - 大量数据只能传递地址
- 带有出口参数的寄存器不能保护和恢复
- 带有入口参数的寄存器可以保护、也可以不保护，但最好能够保持一致



十六进制显示

- 要求：将一个双字数据，以十六进制形式显示
 - 主程序：提供要显示的数据
 - 子程序：将1位十六进制数转换为ASCII码
- 寄存器传递参数
 - 入口参数：AL=1位十六进制数（对应二进制4位）
 - 出口参数：AL=对应数码的ASCII码

少量数据直接传递数值



十进制、二进制和十六进制对应关系表

十进制	二进制	十六进制	十进制	二进制	十六进制
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F



ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符
0	NUT	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	'	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	X	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~



十六进制显示主程序—1

;代码段，主程序

`mov eax,1234abcdh` ;假设一个数据

`xor ebx,ebx` ;相对寻址访问字符串

`mov ecx,8` ;8位十六进制数

`again: rol eax,4` ;高4位循环移位进入低4位

`push eax` ;也可以用`mov edx,eax`

`call htoasc` ;调用子程序`htoasc`

AL低4位传递入口参数

AL传递出口参数
主程序进行保护



十六进制显示主程序-2

`mov regd+4[ebx],al`

`pop eax`

`inc ebx`

`loop again`

`mov eax,offset regd`

`call dispmsg`

AL传递出口参数
主程序进行恢复

;从高位开始保存转换后的ASCII码

;也可以用`mov eax,edx`

;数据段

`regd byte 'EAX=',8 dup(0),'H',0`

+0

+4

`regd+4[ebx]`

;显示

EAX=1234ABCDH

运行结果



十六进制显示原理

- 子程序（HTOASC）实现原理
 - ▶ 数值0~9：加30H 成为字符
'0' ~ '9' 的ASCII码
 - ▶ 数值10~15：加30H，再加7
成为字符'A' ~ 'F' 的ASCII码

ASCII表（部分）

	3	4
0	0	@
1	1	A
2	2	B
3	3	C
4	4	D
5	5	E
6	6	F
7	7	G
8	8	H
9	9	I



十六进制显示子程序—1

;代码段，子程序（解1）

htoasc

proc

and al,0fh ;只取AL的低4位

or al,30h ;AL高4位变成3

cmp al,39h ;是0~9，还是A~F

jbe htoend

add al,7 ;是A~F，ASCII码再加上7

htoend: ret ;子程序返回

htoasc endp



十六进制显示子程序—2

;代码段，子程序（解2）

htoasc

proc

and eax,0fh ;取AL低4位

mov al,ASCII[eax] ;换码

ret

;子程序的局部数据（只读）

ASCII

byte '0123456789ABCDEF'

htoasc

endp



寄存器传递参数

;代码段，主程序

again:

rol eax,4

push eax

call htoasc

mov regd+4[ebx],al

pop eax

inc ebx

loop again

AL

AL

htoasc

;代码段，子程序

proc

and al,0fh

or al,30h

cmp al,39h

jbe htoend

add al,7

htoend: ret

htoasc endp



汇编语言程序设计

共享变量传递参数



共享变量传递参数

- 子程序和主程序使用**同一个变量名**存取数据
- 如果变量定义和使用不在同一个程序模块中，
需要利用**PUBLIC、EXTREN**声明
- 共享变量传递参数，子程序的通用性较差
- 特别适合在多个程序段间、 尤其
在不同的程序模块间传递数据

共享变量对应高级语言的全局变量



二进制输入

- 要求：从键盘以二进制形式输入若干32位数据
 - 主程序：提供保存输入数据的存储空间，保存数据
 - 子程序：输入一个32位二进制数
- 共享变量传递参数
 - 入口参数：无
 - 出口参数：共享变量temp=输入的32位数据

少量数据直接传递数值



二进制输入主程序-1

;数据段

count = 5

array dword count dup(0)

temp dword ?

;共享变量

; 代码段，主程序

mov ecx,count

;输入count个数据

mov ebx,offset array

;EBX间接寻址访问数组



二进制输入主程序-1

; 代码段, 主程序

```
mov ecx, count
```

; 设置循环计数器, 输入count个数据

```
mov ebx, offset array
```

; EBX指向数组首地址, 用于间接寻址访问数组

again:

```
call rdbd
```

; 调用子程序, 输入一个32位二进制数据

```
mov eax, temp
```

; 从共享变量temp中获得子程序的返回值 (出口参数)

```
mov [ebx], eax
```

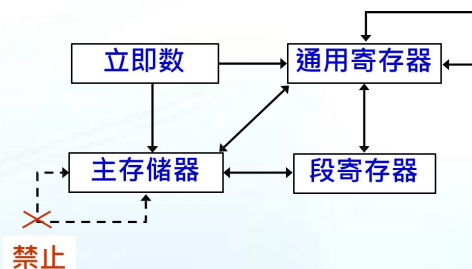
; 将输入的数据存放到数组缓冲区中

```
add ebx, 4
```

; 指针移动到下一个双字位置 (每个数据占4字节)

```
loop again
```

; 循环输入下一个数据



并非任意传送 !

使用共享变量直接传递返回值



二进制输入原理

➤子程序（Rddb）实现原理

- ▶ 输入一个字符，判断是“0”或“1”，减30H，转换成为数值0或1
- ▶ 重复输入字符并转换，每次输入后将前一次数值左移1位， 并与新输入数值合并
- ▶ 输入非0或1的字符、或超出位数，提示错误，重新输入

输入第1位

1

输入前2位

10

输入前3位

101

⋮

输入32位

10101001...11001110



子程序rdbd

; 代码段, 子程序: 输入32位二进制数

```
rdbd proc                ; 子程序开始
    push eax              ; 寄存器保护, 保存EAX
    push ebx              ; 寄存器保护, 保存EBX
    push ecx              ; 寄存器保护, 保存ECX

rdbd1:
    xor ebx, ebx          ; 清零EBX, EBX用于存放二进制结果
    mov ecx, 32           ; 设置循环计数器, 限制输入32个二进制字符

rdbd2:
    call readc            ; 调用子程序输入一个字符
    cmp al, '0'           ; 检测键入字符是否合法 (是否小于'0')
    jb rderr              ; 如果小于'0', 则跳转到出错处理
    cmp al, '1'           ; 检测键入字符是否合法 (是否大于'1')
    ja rderr              ; 如果大于'1', 则跳转到出错处理

    sub al, '0'           ; 对输入的字符进行转换: ASCII码转数值 ('0'→0, '1'→1)
    shl ebx, 1            ; EBX的值乘以2 (左移1位), 为新的二进制位腾出空间
    or bl, al              ; 将新的二进制位加到EBX的最低位 (或运算实现相加)
    loop rdbd2            ; 循环输入下一个字符, 直到输入完32位

    mov temp, ebx         ; 把二进制结果存放到共享变量TEMP中返回给主程序
    call dpccrlf          ; 调用回车换行子程序, 使光标换行
    pop ecx               ; 恢复寄存器ECX
    pop ebx               ; 恢复寄存器EBX
    pop eax               ; 恢复寄存器EAX
    ret                  ; 返回主程序

rderr:
    ; 出错处理标签
    mov eax, offset errmsg ; 显示错误信息
    call dispmsg          ; 调用显示消息子程序
    jmp rdbd1             ; 跳转回开始处, 重新输入

errmsg byte 0dh, 0ah, 'Input error, enter again: ', 0 ; 错误提示信息

rdbd endp                ; 子程序结束
```



二进制输入子程序—1

;代码段，子程序：输入32位二进制数

rdbd

proc

;出口参数：共享变量TEMP

push eax

;寄存器保护

push ebx

push ecx

;EBX用于存放二进制结果

rdbd1: xor ebx,ebx

mov ecx,32

;限制输入字符的个数



二进制输入子程序—2

rdbd2: call readc

;输入一个字符

cmp al,'0'

;检测键入字符是否合法

jb rderr

;不合法则转到出错处理

cmp al,'1'

ja rderr

sub al,'0'

;对输入的字符进行转化

shl ebx,1

;EBX的值乘以2（左移1位）

or bl,al

;BL和AL相加（或）

loop rdbd2

;循环输入字符



二进制输入子程序-3

```
mov temp,ebx      ;把二进制结果存放TEMP返回  
call dispCrLf     ;分行  
pop ecx  
pop ebx  
pop eax  
ret
```

使用共享变量直接传递返回值



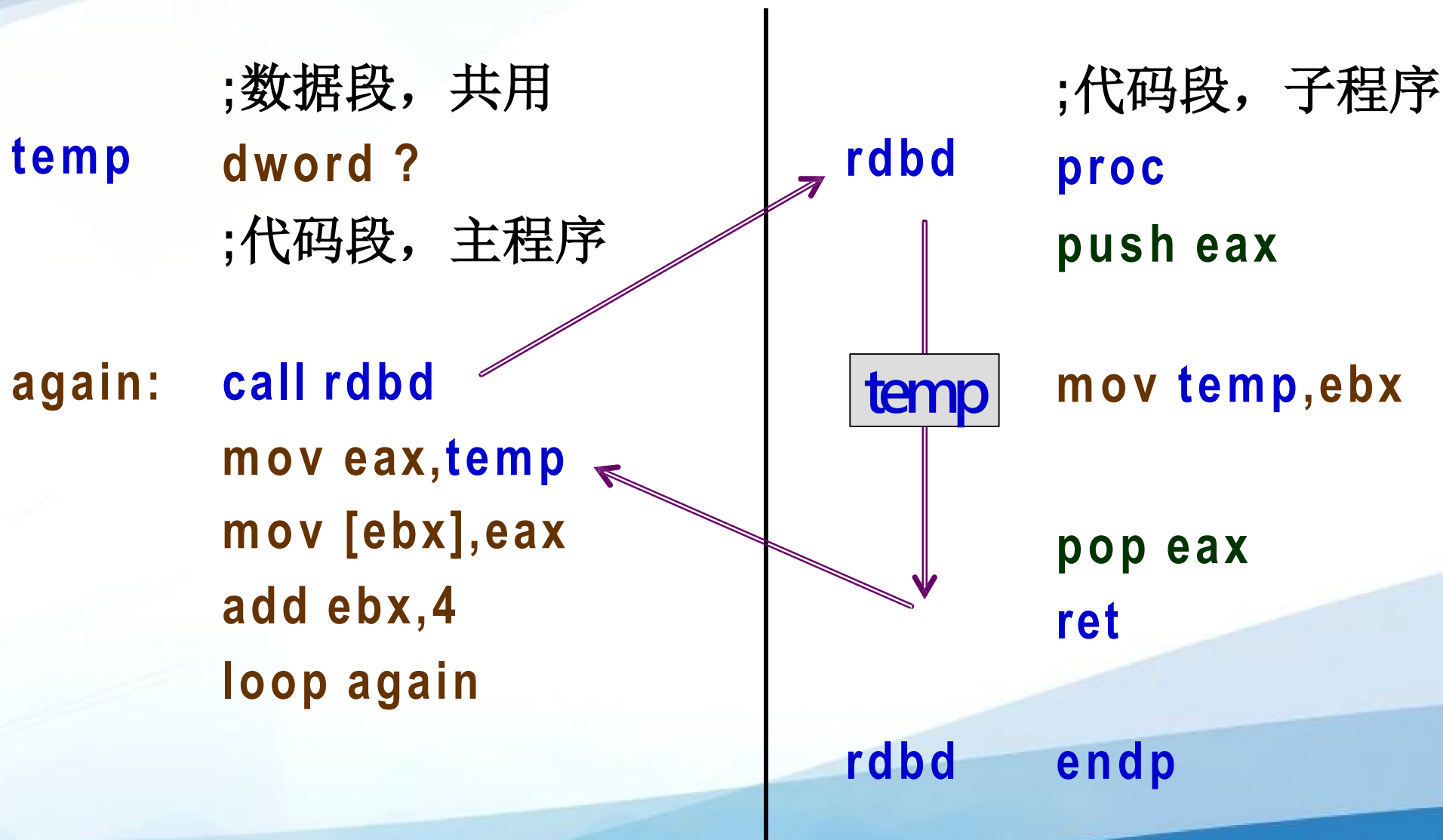
二进制输入子程序—4

```
rderr:    mov eax,offset errmsg    ;显示错误信息
          call dispmsg
          jmp rdbd1                ;重新输入

errmsg    byte 0dh,0ah,'Input error, enter again: ',0
rdbd      endp
```



共享变量传递参数





汇编语言程序设计

堆栈传递参数



堆栈传递参数

- 主程序将入口参数压入堆栈
 - 子程序从堆栈中取出参数
- 采用堆栈传递参数常是程式化的
 - 子程序设置EBP等于当前ESP
 - 利用EBP相对寻址访问堆栈中的参数
- 出口参数通常不使用堆栈传递

高级语言函数调用的参数实质是堆栈传递参数



计算有符号数平均值

- 要求：将数组元素求和，除以元素个数，求得平均值
 - 主程序：提供数组地址和元素个数，显示结果
 - 子程序：求平均值，返回结果
- 堆栈传递入口参数
 - 压入堆栈元素个数（传数值）
 - 压入堆栈数组地址（传地址）
- 寄存器传递出口参数
 - $EAX = \text{平均值}$ （传数值）

堆栈传递数值和地址



主程序rdbd

```
;=====
; 数据段定义
;=====
```

```
.data
array dword 675, 354, -34, 198, 267, 0, 9, 2371, -67, 4257
; 定义包含10个有符号整数的数组
```

```
;=====
; 代码段 - 主程序
;=====
```

```
.code
main proc
    ; 通过堆栈传递参数给子程序
    push lengthof array    ; 压入数据个数(4个字节) - 入口参数1
    push offset array      ; 压入数组地址(4个字节) - 入口参数2

    call mean              ; 调用求平均值子程序
```

```
add esp, 8                ; 主程序平衡堆栈(弹出8个字节); 另一种方式: 子程序可以使用 ret 8 来平衡堆栈
```

```
call dispsid              ; 显示平均值EAX(出口参数)
```

```
; 程序结束
ret
```



子程序

```
mean proc
; 建立堆栈帧
push ebp                ; 保存基址指针
mov ebp, esp            ; 设置基址指针指向当前堆栈顶

; 保护寄存器
push ebx
push ecx
push edx

; 从堆栈中获取参数
; [ebp+8] 指向数组地址 (第一个参数)
; [ebp+12] 指向数据个数 (第二个参数)
mov ebx, [ebp + 8]      ; EBX = 数组地址
mov ecx, [ebp + 12]     ; ECX = 数据个数

; 清零累加和寄存器
xor eax, eax            ; EAX = 0, 用于保存和值
xor edx, edx            ; EDX = 0, 用于数组索引
```

```
mean1:
; 循环累加数组元素
add eax, [ebx + edx * 4] ; 求和: EAX = EAX + array[edx]
add edx, 1               ; 指向下一个数据: edx = edx + 1

; 检查是否已处理完所有元素
cmp edx, ecx             ; 比较当前索引和数据个数
jb mean1                ; 如果edx < ecx, 继续循环

; 计算平均值 (有符号除法)
; 方法1: 使用cdq指令进行符号扩展
cdq                      ; 将EAX符号扩展到EDX:EAX

idiv ecx                 ; 有符号除法指令IDIV功能:
;   EAX ← EDX:EAX ÷ r32/m32 的商
;   EDX ← EDX:EAX ÷ r32/m32 的余数

; 恢复寄存器
pop edx
pop ecx
pop ebx

; 恢复堆栈帧
pop ebp

; 子程序返回
ret ; 注意: 这里主程序负责平衡堆栈 (add esp, 8), 也可以使用 ret 8
; 让子程序平衡堆栈
mean endp
```



计算有符号数平均值主程序

;数据段

array dword 675,354,-34,198,267,0,9,2371,-67,4257

;代码段，主程序

push lengthof array ;压入数据个数(4个字节)

push offset array ;压入数组地址(4个字节)

call mean ;调用求平均值子程序



add esp,8 ;主程序平衡堆栈(弹出8个字节)

call dispsid ;显示平均值EAX(出口参数)



堆栈传递参数

;代码段，主程序

push lengthof array

push offset array

call mean

add esp,8

call dispsid

主程序平衡堆栈

;代码段，子程序

proc

push ebp

mov ebp,esp

...

mov ebx,[ebp+8]

mov ecx,[ebp+12]

...

ret

endp

mean



EAX

mean



计算有符号数平均值子程序—1

;代码段，子程序：计算32位有符号数平均值

mean	proc	;入口参数：顺序压入个数和地址
	push ebp	;出口参数：EAX=平均值
	mov ebp,esp	
	push ebx	;保护寄存器
	push ecx	
	push edx	

采用堆栈传递参数可以程式化

- 子程序设置EBP等于当前ESP
- 利用EBP相对寻址访问堆栈中的参数



计算有符号数平均值子程序—2

mov ebx,[ebp+8]

mov ecx,[ebp+12]

;EBX=取出的数组地址

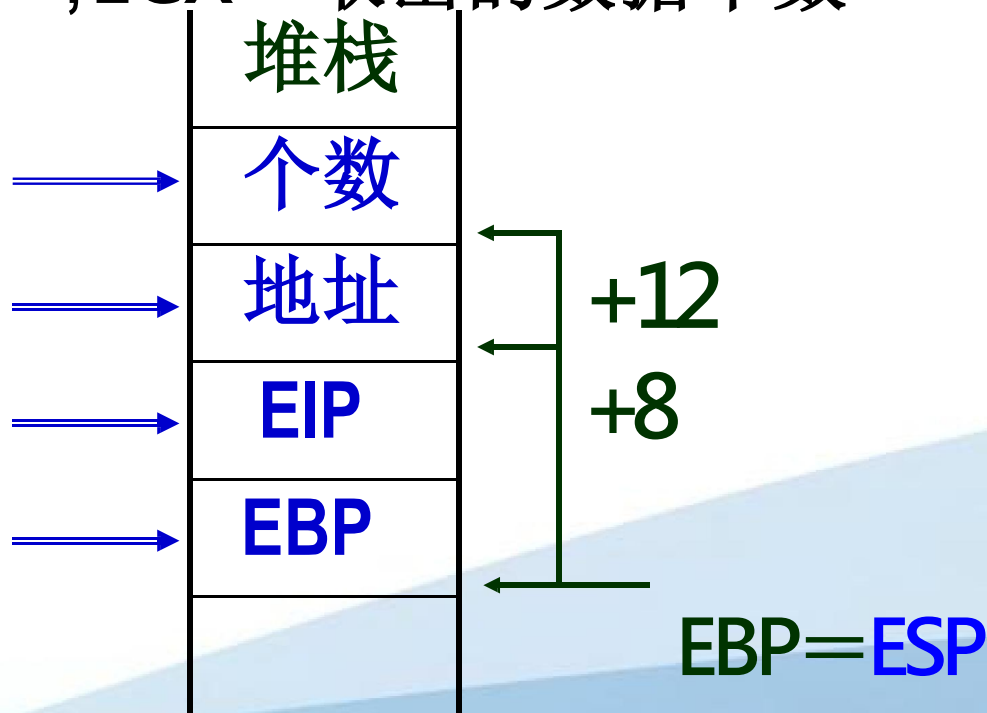
;ECX=取出的数据个数

主程序: **push lengthof array**

主程序: **push offset array**

主程序: **call mean** 子程序:

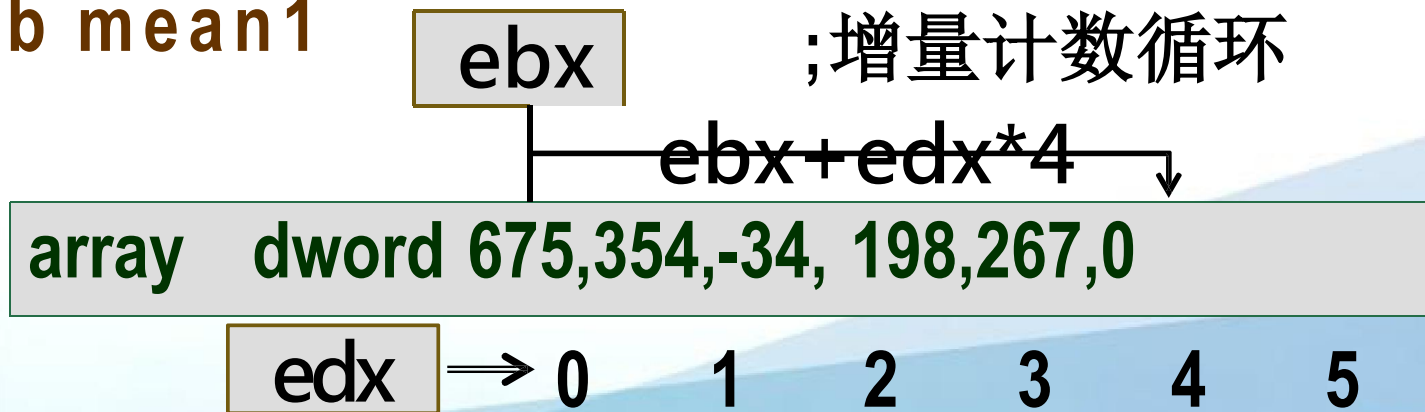
push ebp





计算有符号数平均值子程序—3

xor eax,eax ;EAX保存和值
xor edx,edx ;EDX=指向数组元素
mean1: add eax,[ebx+edx*4] ;求和
add edx,1 ;指向下一个数据
cmp edx,ecx ;比较个数
jb mean1 ;增量计数循环





计算有符号数平均值子程序—4

cdq ;将累加和EAX符号扩展到EDX
;将EAX最高位填入EDX所有位

idiv ecx ;有符号数除法, $EAX = \text{平均值}$

; cdq替代指令
mov edx,eax
sar edx,31

有符号除法:

IDIV src

$EAX \leftarrow EDX$. $EAX \div r32/m32$ 的商

$EDX \leftarrow EDX$. $EAX \div r32/m32$ 的余数



计算有符号数平均值子程序—5

pop edx

;恢复寄存器

pop ecx

堆栈平衡了吗？

pop ebx

add esp,8 ;主程序平衡堆栈

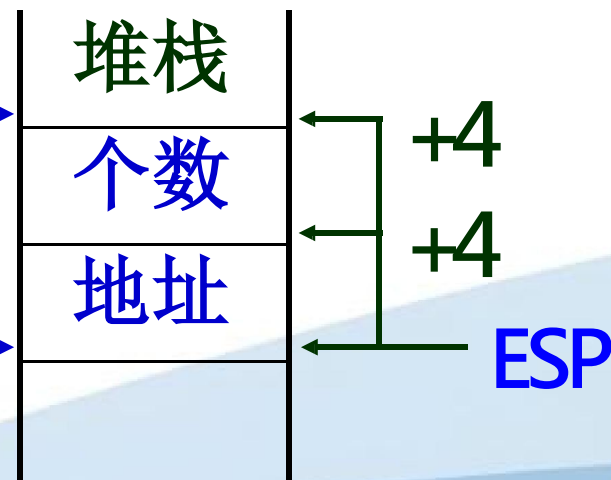
pop ebp

ret

主程序压入参数前ESP

mean endp

子程序返回后ESP



另一种方式：子程序可以使用 ret 8 来平衡堆栈



堆栈传递参数

;代码段，主程序

```
push lengthof array  
push offset array
```

```
call mean
```

```
add esp,8
```

```
call dispsid
```

主程序平衡堆栈

;代码段，子程序

```
proc
```

```
push ebp
```

```
mov ebp,esp
```

...

```
mov ebx,[ebp+8]
```

```
mov ecx,[ebp+12]
```

...

```
ret
```

```
endp
```

mean



mean

EAX



汇编语言程序设计

源文件包含和模块连接



多模块程序结构

- 程序分段、子程序等实现了程序模块化
 - 开发大型应用程序时常使用
 - 多个源程序文件
 - 目标代码模块等
 - 组成完整的程序
- 形成多模块程序结构

MASM支持的多模块方法

- ✓ 源文件包含
- ✓ 模块连接
- ✓ 子程序库
- ✓ 库文件包含



源文件包含

- 大型源程序可以合理地分放在若干个文本文件中
 - ▶ 各种常量定义、声明语句等组织在包含文件 (*.INC)
 - ▶ 常用的或有价值的宏定义存放在宏定义文件 (*.MAC)
 - ▶ 常用的子程序形成汇编语言源文件 (*.ASM)
 - ▶ 任何文本文件

➤ 使用源文件包含伪指令 **INCLUDE**

include 文件名

; 将指定文件内容插入主体源程序文件



存储器数据显示程序—1

;文件名: **eg0508.inc**, 程序的数据段内容

.data

;文件名: **eg0508s.asm**, 子程序

rdhd proc ;十六进制输入子程序

dphd proc ;十六进制输出子程序

write proc ;显示有符号十进制数子程序



存储器数据显示程序—2

;文件名： eg0508. asm， 主程序

include io32.inc ;包含软件I/O 库声明

include eg0508.inc ;包含数据段

.code ;代码段， 主程序

exit 0 ;主程序结束

include eg0508s.asm ;包含子程序代码

end start



源文件包含的使用

➤ 被包含文件

- ▶ 文件名要符合操作系统规范
- ▶ 只能是文本文件
- ▶ 内容被插入源文件包含include语句所在的位置

➤ 实质仍然是一个源程序

- ▶ 只是分开在若干个文件中
- ▶ 只需针对主体源程序文件进行汇编、连接

操作过程

```
make32 eg0508
```



模块连接

➤子程序单独编写一个源程序文件

- ▶ 使用共用伪指令**PUBLIC**和外部伪指令**EXTERN**声明
- ▶ 子程序在代码段，与主程序文件采用相同的存储模型
- ▶ 没有开始执行和结束执行点，但有汇编结束语句
- ▶ 处理好子程序与主程序之间的参数传递问题

子程序源文件汇编为目标模块OBJ文件

连接时输入子程序目标模块文件名

;定义标识符的模块使用

public 标识符 [,标识符]

;调用标识符的模块使用

extern 标识符:类型 [,标识符:类型]



存储器数据显示程序—3

;文件名: eg0508es.asm, 子程序

public rdhd,dphd,write ;子程序共用

extern temp:dword ;外部变量

rdhd proc ;十六进制输入子程序

end ;汇编结束



存储器数据显示程序—4

;文件名: eg0508e.asm, 主程序

temp dword ? ;共享变量定义

extern rdhd:near,dphd:near,write:near ;外部子程序

public temp ;变量共用

.code ;代码段, 主程序

end start



模块连接的操作过程

- 子程序单独编写一个源程序文件
- 子程序源文件汇编形成目标模块OBJ文件
- 连接时输入子程序目标模块文件名

操作过程

```
ML /c /coff eg0508e.asm
```

```
ML /c /coff eg0508es.asm
```

```
LINK32 /subsystem:console eg0508e.obj eg0508es.obj
```



汇编语言程序设计

子程序库和库文件包含



子程序库

- 子程序库是子程序模块的集合，便于统一管理子程序
- 编写存入库文件的子程序
 - 遵循更加严格的子程序模块要求
 - 应该遵循一致的规则（以免在使用时造成混乱）
- 子程序文件编写完成、汇编形成目标模块
- 利用库管理工具程序把子程序模块加入到子程序库



子程序库的使用

- 子程序单独编写一个源程序文件
- 子程序源文件汇编形成目标模块OBJ文件
- 利用库管理工具把子程序模块加入到子程序库
- 在连接主程序时提供子程序库文件名

操作过程

```
ML /c /coff eg0508e.asm
```

```
ML /c /coff eg0508es.asm
```

```
LIB32 /OUT:eg0508.lib eg0508es.obj
```

```
LINK32 /subsystem:console eg0508e.obj eg0508.lib
```



库文件包含

- 要使用已存入库文件中的子程序
- 在主程序源文件中用库文件包含伪指令 INCLUDELIB 声明

➤ 使用库文件包含伪指令 **INCLUDELIB**

include lib 文件名

;使用库文件中的子程序



库文件包含的使用

- 将子程序源文件汇编、模块文件加入子程序库

```
ML /c /coff eg0508es.asm
```

```
LIB32 /OUT:eg0508.lib eg0508es.obj
```

- 源文件中用库文件包含伪指令 **INCLUDELIB** 声明
- 正常对主程序汇编、连接，无需在连接时输入库文件名
 - ▶ 编写主程序、子程序更加独立
 - ▶ 子程序使用更方便

操作过程

```
make32 eg0508e
```



IO32.INC

- 组合源文件包含和库文件包含等方法
 - ▶ 可以精简程序框架，简化程序设计

```
;declare procedures
extern readc:near, readmsg:near
extern disp:near, dispmsg:near, dispctrl:near
...
;declare I/O libraries
includelib io32.lib
```

;源程序框架

include io32.inc



汇编语言程序设计

宏汇编



宏汇编

- 宏（Macro）是具有宏名的一段汇编语句序列
- 宏需要先使用**MACRO/ENDM**伪指令进行定义
- 然后在程序中使用宏名（带参数）进行宏调用

➤ 源程序进行汇编时

宏名被汇编程序用宏定义的
代码序列替代，实现宏展开

- 这个过程就是“宏汇编”

➤ 宏定

~~宏~~名 **macro** [形参表]

;宏定义体

endm



宏定义、宏调用和宏展开

;宏定义

```
WriteString macro msg  
    push eax  
    lea eax,msg  
    call dispmsg  
    pop eax  
endm
```



;宏调用（宏指令）

```
WriteString msg
```



;宏展开

```
    push eax  
    lea eax,msg  
    call dispmsg  
    pop eax
```

```
msg byte 'Hello, Assembly !',13,10,0
```



宏汇编的特点

- 宏需要先定义后使用，且不必在任何段中
 - ▶ 常书写于源程序开始位置
 - ▶ 常用的宏定义可以单独写成一个宏定义文件
- 宏定义中更改了寄存器内容，最好进行保护和恢复
- 宏定义参数灵活，宏定义允许嵌套和递归调用
- 宏调用不需要控制的转移与返回
 - ▶ 宏调用将相应的语句序列复制到宏指令的位置
 - ▶ 宏展开被嵌入源程序，成为一体



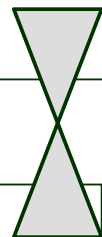
宏与子程序：简化程序

➤ 宏仅是源程序级的简化

- ▶ 宏调用在汇编时进行程序语句的展开，不需要返回
- ▶ 不减小目标程序，执行速度没有改变

➤ 子程序不仅简化源程序，还是目标程序级的简化

- ▶ 子程序调用在执行时由CALL指令转向、RET指令返回
- ▶ 形成的目标代码较短，执行速度减慢





宏与子程序：传递参数

➤ 宏通过形参、实参结合实现参数传递

- ▶ 使用软件方法，简洁直观、灵活多变
- ▶ 传递出错多体现为语法错误，易于发现

➤ 子程序利用寄存器、存储单元或堆栈等实现

- ▶ 运用硬件本身，规则严格、方法固定
- ▶ 传递出错常反映为逻辑或运行错误，较难排除



宏与子程序：选用原则

- ▶ 当程序段较短或要求较快执行时

➤ 选用宏

- ▶ 宏常依附于源程序，适合进行全局性预处理

- ▶ 当程序段较长或为减小目标代码时

➤ 选用子程序

- ▶ 子程序更具有独立性，可以分别编写